If you make that change and re-run the client, you'll see that the connection now uses ChaCha20-Poly1305, so long as both the client and server are using Java 12 or later.

> **WARNING** If you adjust the default parameters, ensure that you set an endpoint identification algorithm. Otherwise Java won't validate that the server's certificate matches the hostname you have connected to and the connection may be vulnerable to man-in-the-middle attacks. You can set the identification algorithm by calling `params.setEndpointIdenticationAlgorithm("HTTPS")`.

AES-CCM is not yet supported by Java, although work is in progress to add support. The Bouncy Castle library (https://www.bouncycastle.org/java.html) supports CCM cipher suites with DTLS, but only through a different API and not the standard `SSLEngine` API. There's an example using the Bouncy Castle DTLS API with CCM in section 12.2.1.

The CCM cipher suites come in two variations:

- The original cipher suites, whose names end in _CCM, use a 128-bit authentication tag.
- Cipher suites ending in _CCM_8, which use a shorter 64-bit authentication tag. This can be useful if you need to save every byte in network messages but provides much weaker protections against message forgery and tampering.